Git 简介

背景

受 git 版本控制的文件夹中包含一个 `.git/`，实际保存 git 的各种对象

- `.git/objects/`：对象存储区
- `.git/refs/`：引用存储区，它包含两个子目录：`heads` 和 `tags`
- `.git/HEAD`：一个指向当前 `HEAD` 的引用
- `.git/config`：仓库的配置文件
- `.git/description`：存放一个供人类阅读的、对该仓库内容的自由格式描述，通常很少使用

配置文件

配置文件非常简单，它是一个类似 INI 格式的文件，举例说明：包含段（`[core]`）和三个字段：

- `repositoryformatversion = 0`：Git 目录格式的版本。0 表示初始格式，1 表示带有扩展的相同格式。如果大于 1，Git 会报错并退出
- `filemode = false`：禁用对工作树中文件模式（权限）变更的跟踪
- `bare = false`：表示该仓库包含一个工作树。Git 支持一个可选的 worktree 键，用于指定工作树的位置（如果不是..）

Git 中的对象

Git 对象究竟是什么？从核心上讲，Git 是一个“内容寻址的文件系统”。这意味着，与普通文件系统中文件名是任意的且与文件内容无关不同，Git 存储的文件名是通过数学方法从文件内容推导出来的。这有一个非常重要的含义：如果一个文本文件的单个字节发生了变化，它的内部名称也会随之改变。简单来说：在 Git 中你不会修改一个文件，而是在不同的位置创建一个新文件。对象就是这样：Git 仓库中的文件，其路径由它们的内容决定。

Git 存储给定对象的路径是通过计算其内容的 SHA-1 哈希值来确定的。更精确地说，Git 将哈希值呈现为小写的十六进制字符串，并将其分成两部分：前两个字符和剩余部分。它使用第一部分作为目录名，剩余部分作为文件名（这是因为大多数文件系统不喜欢单个目录中有太多文件，否则会慢如蜗牛。Git 的方法创建了 256 个可能的中间目录，从而将每个目录的平均文件数除以 256）。

Git 有四种对象类型：blob、commit、tag 和 tree

Git 对象的格式

一个对象以一个指定其类型的头部开始：blob、commit、tag 或 tree。这个头部后面跟着一个 ASCII 空格（0x20），然后是对象大小（以 ASCII 数字表示的后面跟着的对象内容的字节数），接着是空字节（0x00），最后是对象的内容。一个提交对象的前 48 个字节看起来像这样：

```
00000000  63 6f 6d 6d 69 74 20 31  30 38 36 00 74 72 65 65  |commit 1086.tree|
00000010  20 32 39 66 66 31 36 63  39 63 31 34 65 32 36 35  | 29ff16c9c14e265|
00000020  32 62 32 32 66 38 62 37  38 62 62 30 38 61 35 61  |2b22f8b78bb08a5a|
```

在第一行中，我们看到类型头部、一个空格（0x20）、ASCII 格式的大小（1086）和空字节分隔符 0x00。第一行最后四个字节是该对象内容的开始，即单词 “tree”——我们将在讨论提交时进一步讨论。
对象（头部和内容）使用 zlib 压缩存储。

blob 对象

```
blob <size>\0<content>
```

blob 对象

```
blob <size>\0<content>

62 6c 6f 62 20 31 32 00 68 65 6c 6c 6f 20 77 6f 72 6c 64 0a
└──────────┘ ␣ └────┘ ␣ └──────────────────────────────────┘
  "blob"    " "  "12" \0            "hello world\n"
```

blob 对象

blob <size>\0<content>

```
62 6c 6f 62 20 31 32 00 68 65 6c 6c 6f 20 77 6f 72 6c 64 0a
└──────────┘ ␣ └────┘ ␣ └──────────────────────────────────┘
  "blob"    " "  "12" \0            "hello world\n"
```

```
import hashlib

raw = b"blob␣12\0hello␣world\n"
sha = hashlib.sha1(raw).hexdigest()
# → "3b18e512dba79e4c8300dd08aeb37f8e728b8dad"  (40 位十六进制)
```

blob 对象

blob <size>\0<content>

```
62 6c 6f 62 20 31 32 00 68 65 6c 6c 6f 20 77 6f 72 6c 64 0a
└──────────┘ ␣ └────┘ ␣ └──────────────────────────────────┘
  "blob"    " "  "12" \0            "hello world\n"
```

```
import hashlib

raw = b"blob␣12\0hello␣world\n"
sha = hashlib.sha1(raw).hexdigest()
# → "3b18e512dba79e4c8300dd08aeb37f8e728b8dad"  (40 位十六进制)
```

```
import zlib

raw = b"blob␣12\0hello␣world\n"
compressed = zlib.compress(raw)

# 写入 .git/objects/ab/3c8d...
with open(".git/objects/3b/18e512dba79e4c8300dd08aeb37f8e728b8dad", "wb") as f:
    f.write(compressed)
```

commit 对象

一个提交对象（未压缩，不含头部）的内容大致如下：

```
tree 29ff16c9c14e2652b22f8b78bb08a5a07930c147
parent 206941306e8a8af65b66eaaaea388a7ae24d49a0
author Thibault Polge <thibault@thb.lt> 1527025023 +0200
committer Thibault Polge <thibault@thb.lt> 1527025044 +0200
gpgsig -----BEGIN PGP SIGNATURE-----

 iQIzBAABCAAdFiEExwXquOM8bWb4Q2zVGxM2FxoLkGQFAlsEjZQACgkQGxM2FxoL
 kGQdcBAAqPP+ln4nGDd2gETXjvOpOxLzIMEw4A9gU6CzWzm+oB8mEIKyaH0UFIPh
 rNUZ1j7/ZGFNeBDtT55LPdPIQw4KKlcf6kC8MPWP3qSu3xHqx12C5zyai2duFZUU
 wqOt9iCFCscFQYqKs3xsHI+ncQb+PGjVZA8+jPw7nrPIkeSXQV2aZb1E68wa2YIL
 3eYgTUKz34cB6tAq9YwHnZpyPx8UJCZGkshpJmgtZ3mCbtQaO17LoihnqPn4UOMr
 V75R/7FjSuPLS8NaZF4wfi52btXMSxO/u7GuoJkzJscP3p4qtwe6Rl9dc1XC8P7k
 NIbGZ5Yg5cEPcfmhgXFOhQZkDOyxcJqBUcoFpnp2vu5XJl2E5I/quIyVxUXi6O6c
 /obspcvace4wy8uOObdVhc4nJ+Rla4InVSJaUaBeiHTW8kReSFYyMmDCzLjGIu1q
 doU61OM3Zv1ptsLu3gUE6GU27iWYj2RWN3e3HE4Sbd89IFwLXNdSuM0ifDLZk7AQ
 WBhRhipCCgZhkj9g2NEk7jRVslti1NdN5zoQLaJNqSwO1MtxTmJ15Ksk3QP6kfLB
 Q52UWybBzpaP9HEd4XnR+HuQ4k2K0ns2KgNImsNvIyFwbpMUyUWLMPimaV1DWUXo
 5SBjDB/V/W2JBFR+XKHFJeFwYhj7DD/ocsGr4ZMx/lgc8rjIBkI=
 =lgTX
 -----END PGP SIGNATURE-----

Create first draft
```

这种格式是 RFC 2822 中定义的邮件消息格式的简化版本。它以一系列键值对开始，空格作为键/值分隔符，最后是提交消息，消息可以跨越多行。值可以延续多行，后续行以空格开头，解析器必须去掉这个空格（如上面的 gpgsig 字段，它跨越了 16 行）。

commit 对象

让我们来看看这些字段：

- tree：指向一个树（tree）对象的引用。树对象将 blob ID 映射到文件系统位置，描述工作树的一个状态。简单来说，它就是提交的实际内容：文件内容及其存放位置。
- parent：指向此提交的父提交的引用。这个字段可以重复：例如合并提交就有多个父提交。它也可以不存在：仓库中的第一个提交显然没有父提交。
- author 和 committer：它们是分开的，因为提交的作者不一定是有权提交它的人（这对 GitHub 用户来说可能不太明显，但很多项目通过电子邮件进行 Git 操作）。
- gpgsig：此对象的 PGP 签名。

备注：对象等同性规则

Git 关于对象等同性有两条严格规则：

1. 第一条规则是：相同的名字总是引用相同的对象。我们已经看到这一点，这只是对象名字是其内容哈希值的一个结果。
2. 第二条规则略有不同：相同的对象总是被相同的名字引用。这意味着不应该存在两个不同的名字指向等价的对象。这就是字段顺序重要的原因：通过修改给定提交中字段出现的顺序（例如把 tree 放在 parent 之后），我们会改变该提交的 SHA-1 哈希值，从而创建两个等价但数值上不同的提交对象。

例如，在比较树时，Git 会假定两个不同名字的树是不同的——这就是为什么我们必须确保树对象的元素被正确排序，以免产生不同但等价的树。

commit 对象

```
commit <size>\0<kvlm-content>
```

commit 对象

commit <size>\0<kvlm-content>

kvlm（Key-Value List with Message）格式

```
<key> <value>\n
...
<key> <value>\n
\n
< message >
```

- 每行一个 key value 对，以换行分隔
- 空行（连续两个 n）分隔头部和消息体
- 消息体可以包含任意多行文本
- 值可以跨多行（续行以空格开头）

tree 对象

```
tree <size>\0<entries...>
```

tree 对象

```
tree <size>\0<entries...>
```

<entries...> 是一系列变长条目，单个条目格式为

```
    <mode> <path>\0<20-byte SHA-1>
```

tree 对象

tree <size>\0<entries...>

<entries...> 是一系列变长条目，单个条目格式为

    <mode> <path>\0<20-byte SHA-1>

```
project/
├── README.md
└── src/
    └── main.py
```

```
tree 72\0100644 README.md\0\x01\x23\x45\x67\x89\xab\xcd\xef\x01\x23\x45\x67\x89\xab\
    xcd\xef\x01\x23\x45\x67040000 src\0\x89\xab\xcd\xef\x01\x23\x45\x67\x89\xab\xcd\
    xef\x01\x23\x45\x67\x89\xab\xcd\xef\x01\x23
```

tree 对象

tree <size>\0<entries...>

<entries...> 是一系列变长条目，单个条目格式为

    <mode> <path>\0<20-byte SHA-1>

```
project/
├── README.md
└── src/
    └── main.py
```

```
tree 72\0100644 README.md\0\x01\x23\x45\x67\x89\xab\xcd\xef\x01\x23\x45\x67\x89\xab\
    xcd\xef\x01\x23\x45\x67040000 src\0\x89\xab\xcd\xef\x01\x23\x45\x67\x89\xab\xcd\
    xef\x01\x23\x45\x67\x89\xab\xcd\xef\x01\x23
```

注意到，只有两个条目，src 对应的是一个 tree 对象，main.py 包含在这个 tree 对象中.

sha 是 20 字节二进制 SHA-1（非十六进制）

路径名不包含父目录，只存当前层级名称

引用（ref）

引用位于 `.git/refs` 的子目录中，是包含对象哈希的十六进制表示（以 ASCII 编码）的文本文件

```
6071c08bcb4757d8c89a30d9755d2466cef8c1de
```

引用也可以引用另一个引用，从而间接地指向一个对象，这时它们看起来像这样：

```
ref: refs/remotes/origin/master
```

标签作为引用

引用最简单的用途是标签。标签只是用户为对象（通常是提交）定义的名称。标签的一个非常常见的用途是标识软件版本：假设你刚刚合并了程序版本 12.78.52 的最后一个提交，那么你最近的提交（我们称它为 6071c08）就是你的版本 12.78.52。要显式地建立这个关联，你只需执行：

```
git tag v12.78.52 6071c08
# the object hash ^here^^ is optional and defaults to HEAD.
```

这会创建一个名为 v12.78.52 的新标签，指向 6071c08。标签类似于别名：它引入了一种引用现有对象的新方式。标签创建后，名称 v12.78.52 就指向了 6071c08。例如，以下两个命令现在完全等价：

```
git checkout v12.78.52
git checkout 6071c08
```

备注

版本是标签的常见用途，但像 Git 中的几乎所有东西一样，标签没有预定义的语义：它们意味着你想要的任何含义，可以指向任何你想要的对象，你甚至可以给 blob 打标签！

标签

标签实际上就是引用。它们位于 `.git/refs/tags/` 层级结构中。唯一值得注意的一点是，它们有两种形式：轻量级标签和标签对象。

- “轻量级” 标签只是指向提交、树或 blob 的普通引用。
- 标签对象是指向 tag 类型对象的普通引用。与轻量级标签不同，标签对象有作者、日期、可选的 PGP 签名和可选的附注。它们的格式与提交对象相同。

分支

什么是分支？答案实际上出奇地简单，但也可能只是让人惊讶：分支是指向提交的引用。你甚至可以说分支是提交的一种名称。在这方面，分支与标签完全相同。标签是位于 `.git/refs/tags` 中的引用，分支是位于 `.git/refs/heads` 中的引用。
当然，分支和标签之间有一些区别：

- 标签可以指向任何对象，而分支是指向提交的引用；
- 最重要的是，分支引用在每次提交时都会更新。这意味着每当你提交时，Git 实际上会执行以下操作：
  1. 创建一个新的提交对象，以当前分支的 ID（提交的 ID！）作为其父提交；
  2. 提交对象被哈希并存储；
  3. 分支引用被更新以指向新提交的哈希值。

就是这样。
那么当前分支呢？实际上更简单。它是一个位于 refs 层级之外的引用文件，在 `.git/HEAD` 中，它是一个间接引用（即它的形式是 `ref: path/to/other/ref`，而不是一个简单的哈希）。

备注

分离的 HEAD
当你只是检出一个随机的提交时，Git 会警告你处于“分离的 HEAD 状态”。这意味着你不再在任何分支上。在这种情况下，`.git/HEAD` 是一个直接引用：它包含一个 SHA-1。

暂存区和索引文件

要在 Git 中提交，首先需要使用 `git add` 和 `git rm` 来“暂存”一些更改，然后才提交这些更改。上次提交和下次提交之间的这个中间阶段称为暂存区。
用提交对象或树对象来表示暂存区似乎是自然的，但 Git 实际上使用了一种完全不同的机制，即所谓的索引文件。
在提交之后，索引文件是该提交的一种副本：它保存与对应树相同的路径/blob 关联。但它还保存了关于工作树中文件的额外信息，比如它们的创建/修改时间，这样 git status 通常不需要实际比较文件：它只需检查文件的修改时间是否与索引文件中存储的相同，只有当不同时才执行实际比较。
因此，你可以把索引文件看作一个三向关联列表：不仅将路径与 blob 关联，还将路径与实际的文件系统条目关联。

1. 当仓库处于“干净”状态时，索引文件包含与 HEAD 提交完全相同的内容，加上关于相应文件系统条目的元数据。例如，它可能包含类似这样的信息：
   There's a file called src/disp.c whose contents are blob 797441c76e59e28794458b39b0f1eff4c85f4fa0. The real src/disp.c file, in the worktree, was created on 2023-07-15 15:28:29.168572151, and last modified 2023-07-15 15:28:29.1689427709. It is stored on device 65026, inode 8922881.
2. 当你执行 git add 或 git rm 时，索引文件会被相应修改。在上面的例子中，如果你修改了 src/disp.c 并添加了更改，索引文件将更新为新的 blob ID（当然，blob 本身也会在这个过程中被创建），并且各种文件元数据也会被更新，以便 git status 知道何时不需要比较文件内容。
3. 当你提交这些更改时，会从索引文件生成一个新的树，用该树生成一个新的提交对象，分支被更新，完成操作。

备注

关于术语
暂存区和索引是同一回事，但“暂存区”更像是 Git 面向用户的功能名称（本可以用其他方式实现），是抽象概念；而“索引文件”特指这个抽象功能在 Git 中的实际实现方式。

索引文件.git/index

```
┌──────────┬──────────┬──────────────────┬──────────────────┐
│ HEADER   │ ENTRIES  │ EXTENSIONS (可选)│ SHA-1 checksum   │
│ 12 bytes│ 变长     │ 变长             │ 20 bytes         │
└──────────┴──────────┴──────────────────┴──────────────────┘
```

- 头部（Header，12 字节）

字段	大小	说明
签名	4 字节	`DIRC`（DirCache）
版本号	4 字节	大端整数，通常为 2
条目数	4 字节	大端整数

- 一系列条目，已排序，每个代表一个文件，填充到 8 字节的倍数

```
┌──────────┬──────────┬──────────┬──────────┬──────────┬──────────┬──────────┬──────────┐
│ ctime    │ mtime    │ dev      │ ino      │ mode     │ uid      │ gid      │ fsize    │
│ 8 bytes │ 8 bytes│ 4 bytes│ 4 bytes  │ 4 bytes│ 4 bytes│ 4 bytes│ 4 bytes│
├──────────┼──────────┼──────────┼──────────┤
│ SHA-1    │ flags    │ name     │ padding  │
│ 20 bytes│ 2 bytes│ 变长     │ 0-7 bytes│
└──────────┴──────────┴──────────┴──────────┘
```

- 扩展数据（Extensions，可选）版本 2 支持可选扩展，位于条目之后、校验和之前
- SHA-1 校验和（20 字节）整个索引文件（除最后 20 字节外）的 SHA-1 哈希，用于完整性验证

Git 索引中不包含文件夹条目，只包含文件（blob）的条目

索引条目：

```
README.md
src/main.py
src/utils/helper.py
```

隐含的目录结构：

```
.
├── README.md
└── src/
    ├── main.py
    └── utils/
        └── helper.py
```

git 的 ignore 规则

ignore 规则分为两种，特定于某一个目录的 (scoped) 和全局的 (absolute)

```
| 来源                  | 类型     | 优先级 | 说明           |
| ---------------------| -------- | ------ | ---------------|
| .git/info/exclude    | absolute | 中     | 仓库本地排除   |
| ~/.config/git/ignore | absolute | 低     | 用户全局配置   |
| 索引中的 .gitignore  | scoped   | 高     | 按目录层级生效 |
```

.git 所在文件夹为 project，那么可能有文件 project/.gitignore
路径：project/src/support/w32/legacy/sound.c~

检查顺序：
1. src/support/w32/legacy/.gitignore
2. src/support/w32/.gitignore
3. src/support/.gitignore
4. src/.gitignore
5. .gitignore
6. .git/info/exclude
7. ~/.config/git/ignore

要注意的是，在检测是否满足某一特定 ignore 文件中的规则时，要遍历全部规则，不是短路返回。因为后面的规则可能否定前面的。只要满足某个 ignore 文件中的规则，不会再向上查找，马上返回。如果所有文件都不匹配，默认不被忽略。还有一点要注意，我们是使用 index 中已经存在的.gitignore 文件。